Design Review Guidelines

Design Review Checklist

1. Upstream and Dependencies

- Verify upstream endpoints are correct and aligned with intended integration points.
- Confirm endpoint ownership (Hugo, BOSS, Apollo, etc.) and routing:
  - `hi/apollo/` → APIs directly hitting Apollo
  - `hi/hugo/` → APIs directly hitting Hugo
- URIs **must not include version numbers** (e.g. `v1`, `v2`) for this Hugo cap mod project.

---

2. Endpoint Design

- **RESTful endpoints**
  - Must use appropriate identifiers in URI (`membershipId`, `personId`, etc.).
  - 404 allowed **only** when a true resource lookup fails (e.g. `GET /customers/{id}`).
- **Non-RESTful endpoints**
  - Must justify why they can't be RESTful.
  - Should not contain identifiers in path — move them to:
    - Query params (for `GET` / `DELETE`)
    - Request body (for `POST` / `PUT`)
  - Example:
    ❌ `/hi/memberships/services/{membershipId}/eclipse-eligibility-history`
    ✅ `/hi/memberships/services/eclipse-eligibility-history?membershipId=123`

---

3. Request/Response and Domain Model

- **For all RESTful endpoints** Request and Response types must **match the domain model**.
- Simplify or refactor complex/bloated response objects.
- Ensure **response codes** follow standard usage:
  - Discuss any introduction of `404`, `422`, `429` with the broader group.
  - Prefer `200 OK` with empty body for "no data" scenarios (instead of `404`).

- Return created/updated resource or identifiers for `POST` and `PUT`.

---

4. Request and Validation

- **Query Params & Fields**
  - Enum values must be listed in *Field Info* table.
  - Example values or field descriptions should be documented.
- **Request Validation**
  - Path params are structural — do not add validation rules for missing path params.
  - Confirm correct handling of query/body parameters.

---

5. HTTP Methods

GET

- Validate query/path params and URI consistency.
- Allowed response codes: `200, 400, 401, 403`.
- No business validation rules expected — if needed, discuss first.
- Ensure proper authorization headers (e.g. session tokens).

POST

- Used only for **create** operations (not disguised GETs).
- Requires session token.
- Return created object or identifiers.
- Response codes: `201, 400, 401, 403, 409`.
- `200` allowed only for “read” style POSTs (must be marked as read scope).

PUT

- Used only for **update** of existing resources.
- Identifier must be in URI.
- Requires session token.
- Return updated object or identifiers.
- Response codes: `200, 400, 401, 403`.
- Idempotent requests (same body) should return `200` with request body echoed back *(to be confirmed by Anbu)*.

DELETE

- Used only for true resource deletions.

- Identifier required in URI.
- Requires session token.
- Response codes: `204, 400, 401, 403`.

---

6. Headers & Auth

- Confirm which header is standardised: `SessionToken` is the keyword.
- Check whether `Ocp-Apim-Subscription-Key` is required.
- Authorization requirements must be explicitly captured.

---

7. Confluence Hygiene

- All **comments must be resolved** before development begins.
- Ensure sample requests and environment setup are usable (refer DotNet Team Confluence for examples).

---

8. Naming & Conventions

- Rename any `lanId` or `bossLanId` → `bossUserId`.
- Maintain consistent casing and parameter naming conventions.
- Json Request/Response types should have “@” before types